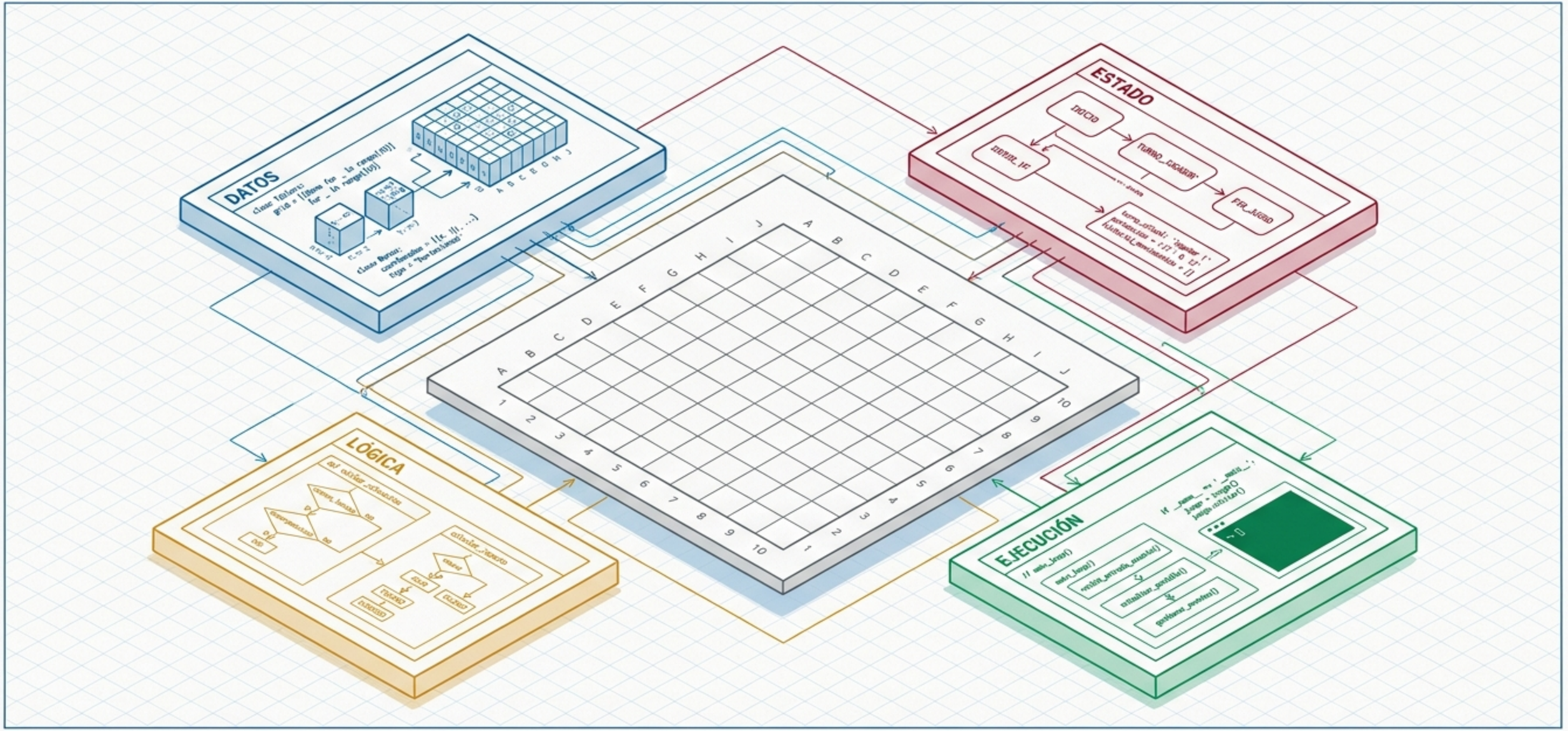


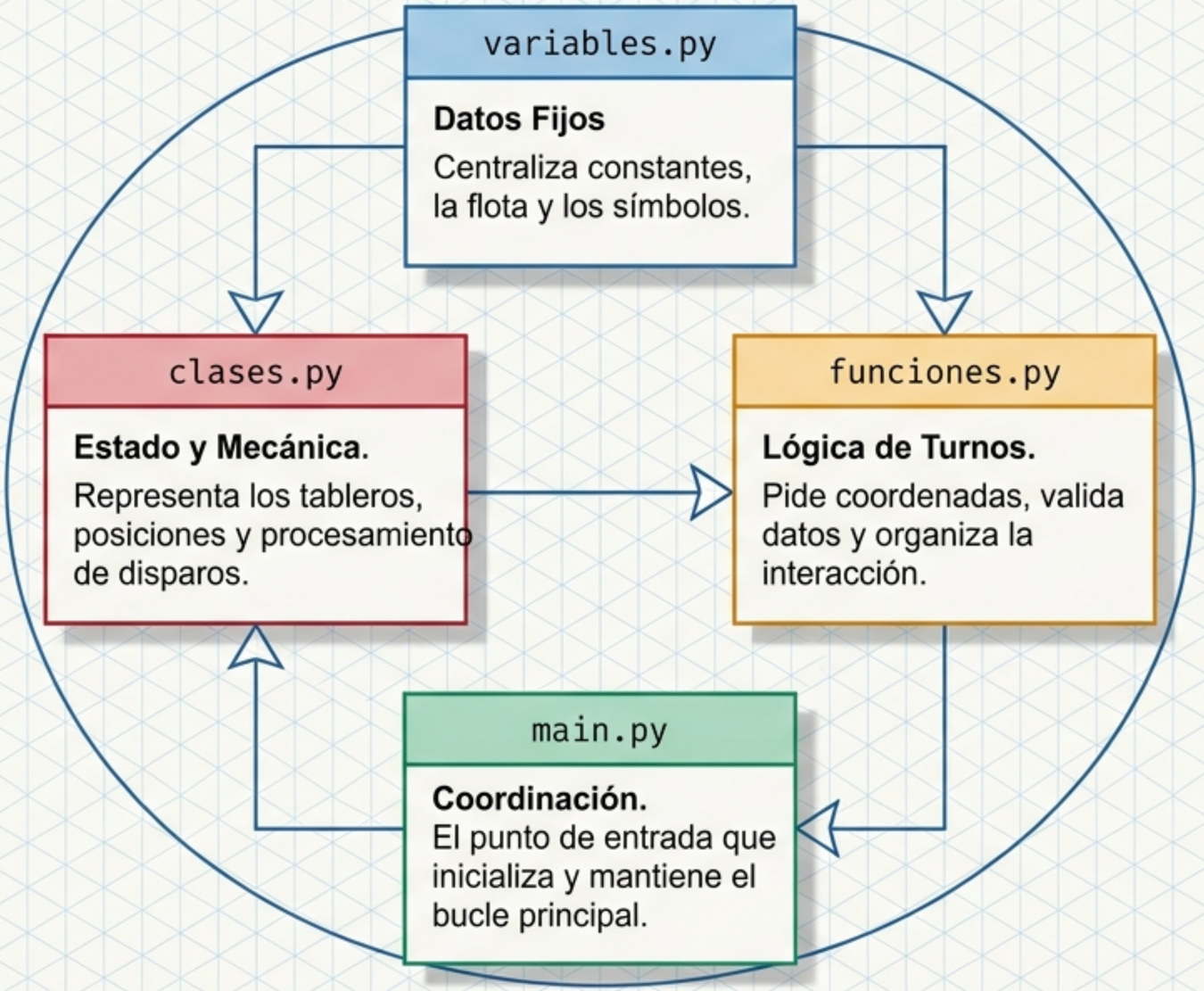
Arquitectura de «Hundir la Flota» en Python

Diseño modular, lógica de estados y flujo de ejecución.

Un análisis estructural basado en el principio de separación de responsabilidades.

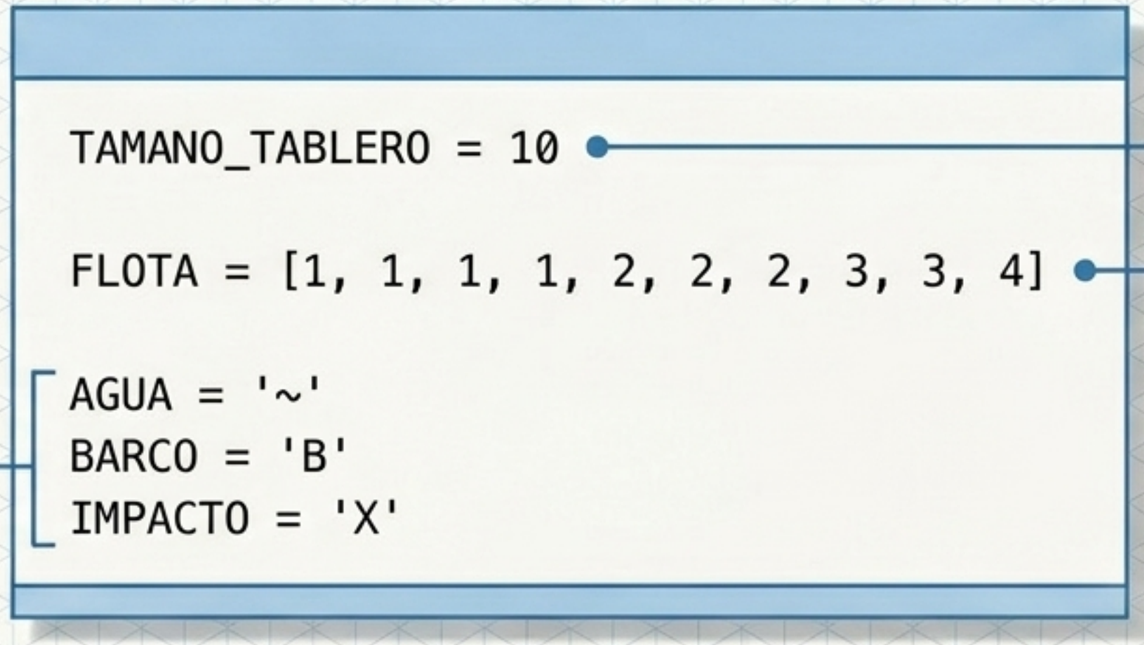
Gemini Notebook

El Modelo de 4 Pilares: Separación de Responsabilidades



Gemini Notebook

La Fuente de la Verdad (Datos Fijos)



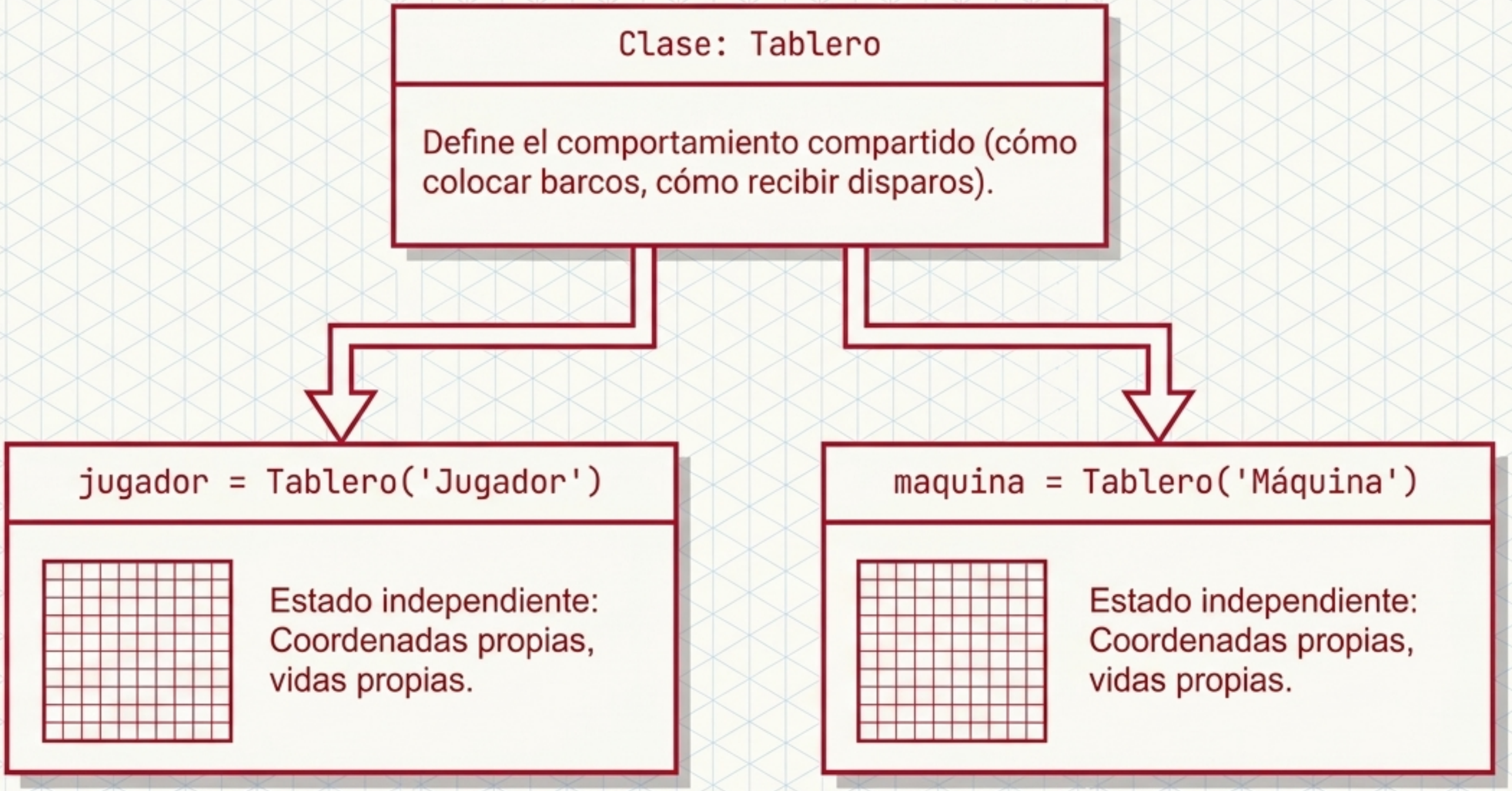
Escalabilidad.
Permite cambiar el tamaño de la cuadrícula en un solo lugar sin reescribir la lógica de validación.

Diccionario visual.
Aisla la representación gráfica de la lógica interna.

Generación automatizada.
Almacena 10 barcos (20 vidas). Permite iterar la colocación con un simple "for eslora in FLOTA:"

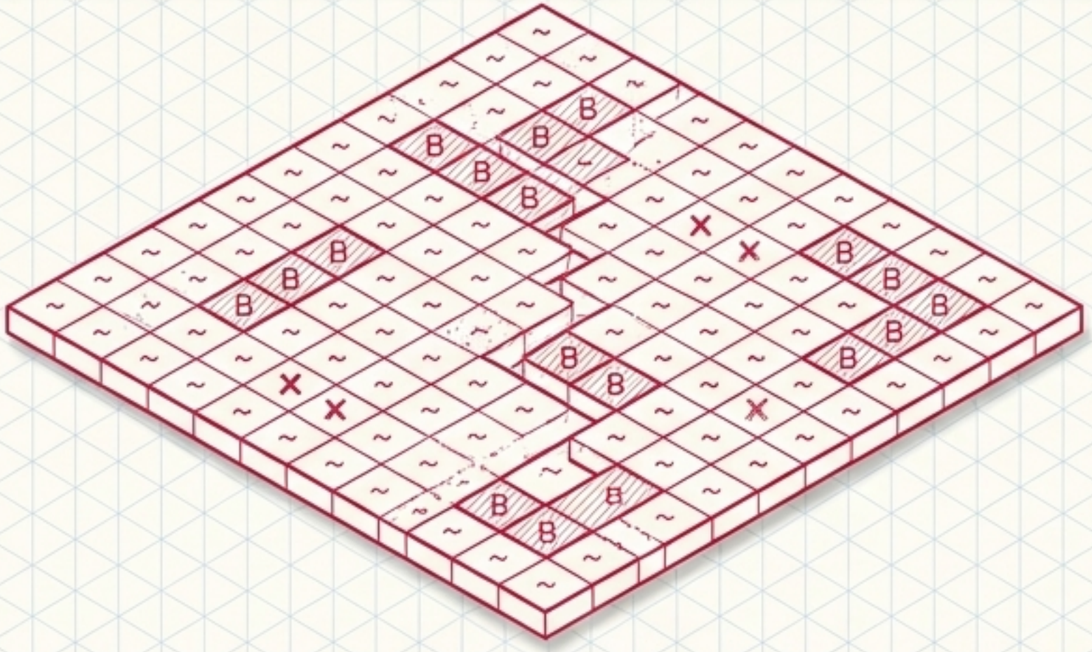
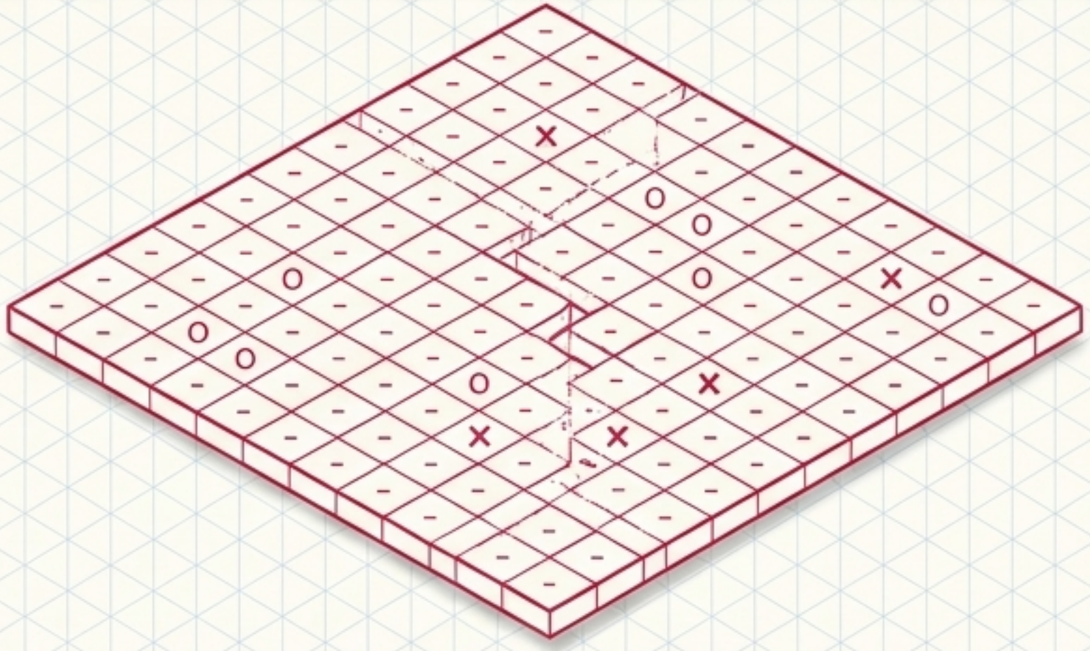
Gemini Notebook

El Paradigma de Objetos: Un Molde, Dos Entidades



Gemini Notebook

El Problema de la Doble Matriz

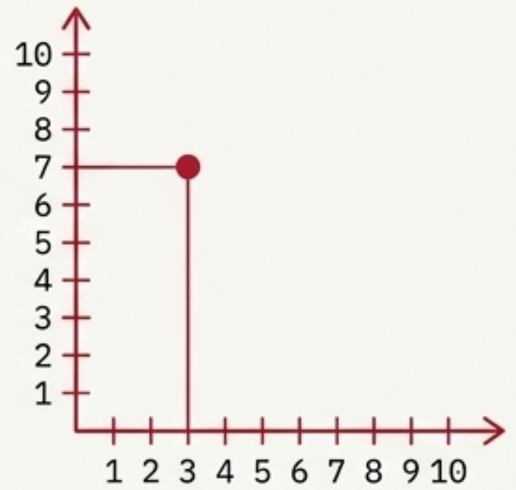
self.tablero	self.disparos
	
Función: Dónde están mis barcos y qué me han disparado. **Estado Inicial:** Lleno de AGUA (~).	**Función:** El radar. Registra mis disparos contra el rival. **Estado Inicial:** Lleno de SIN_DISPARAR (-).

Conclusión: Si guardáramos todo en una sola matriz, mezclaríamos nuestra flota oculta con la inteligencia de los barcos enemigos.

Gemini Notebook

Traducción Espacial: Del Humano a Python

Input Humano



Plano cartesiano (1 a 10)

Conversión (-1)

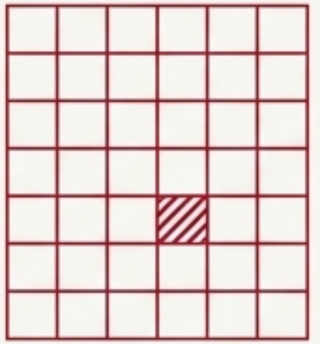
`fila = y - 1`
`(7 - 1 = 6)`

`columna = x - 1`
`(3 - 1 = 2)`

Las listas en Python comienzan en índice 0.

Acceso a Matriz

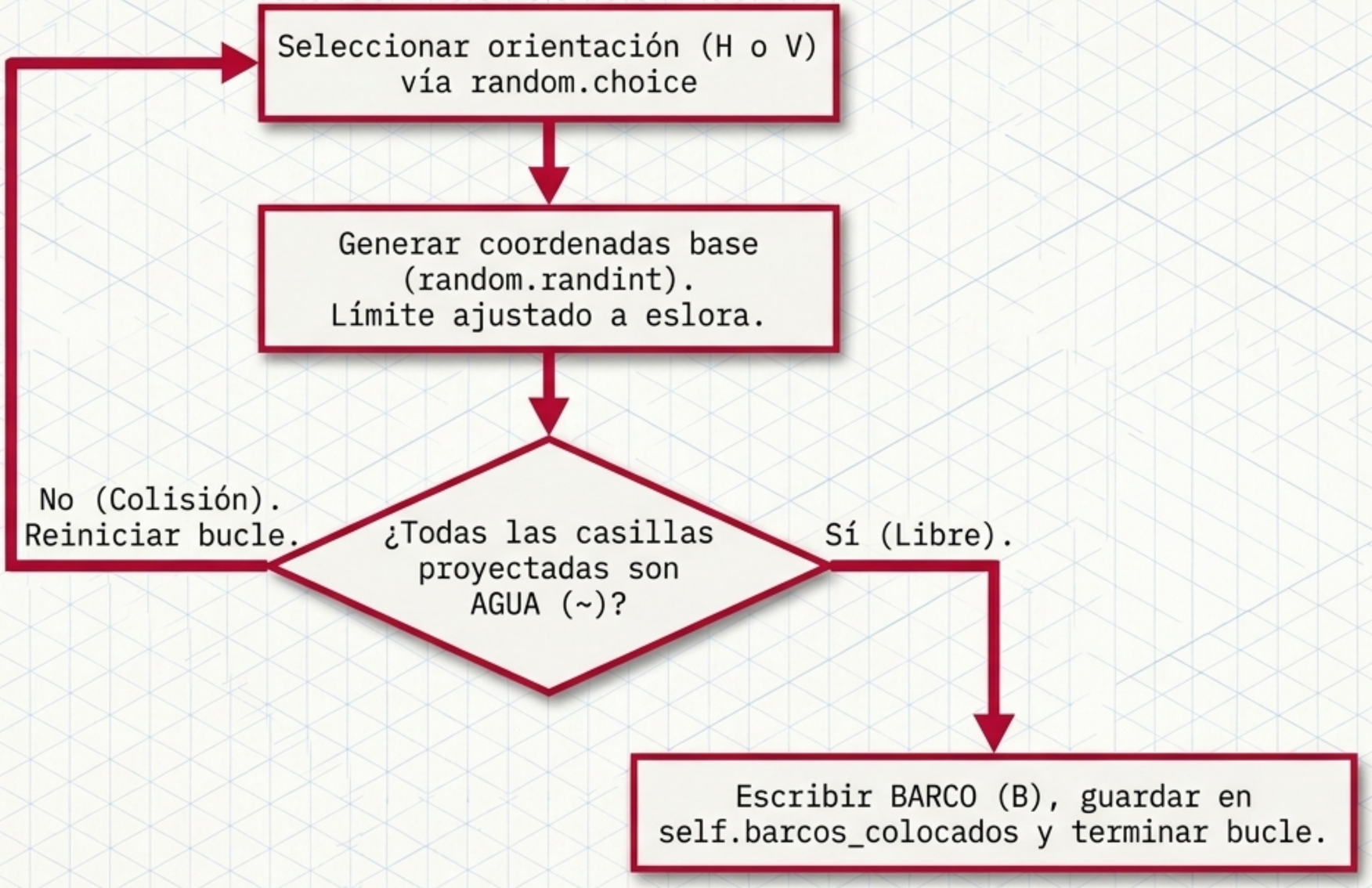
`self.tablero[6][2]`



Inversión de ejes: Fila Fila (lista exterior) primero, columna (lista interior) después.

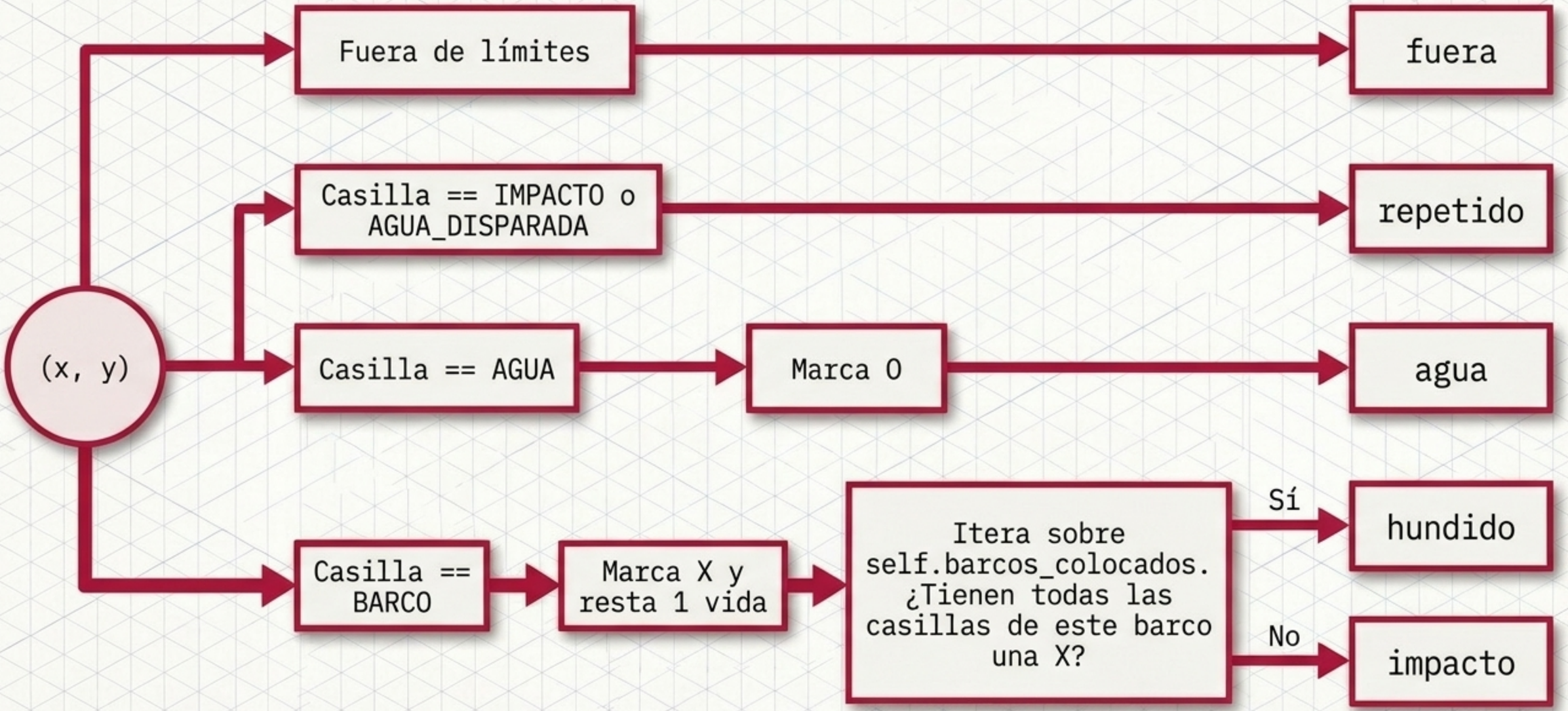
Gemini Notebook

El Motor de Colocación (Generación Aleatoria)



Gemini Notebook

El Algoritmo de Disparo (Lógica de Impacto)



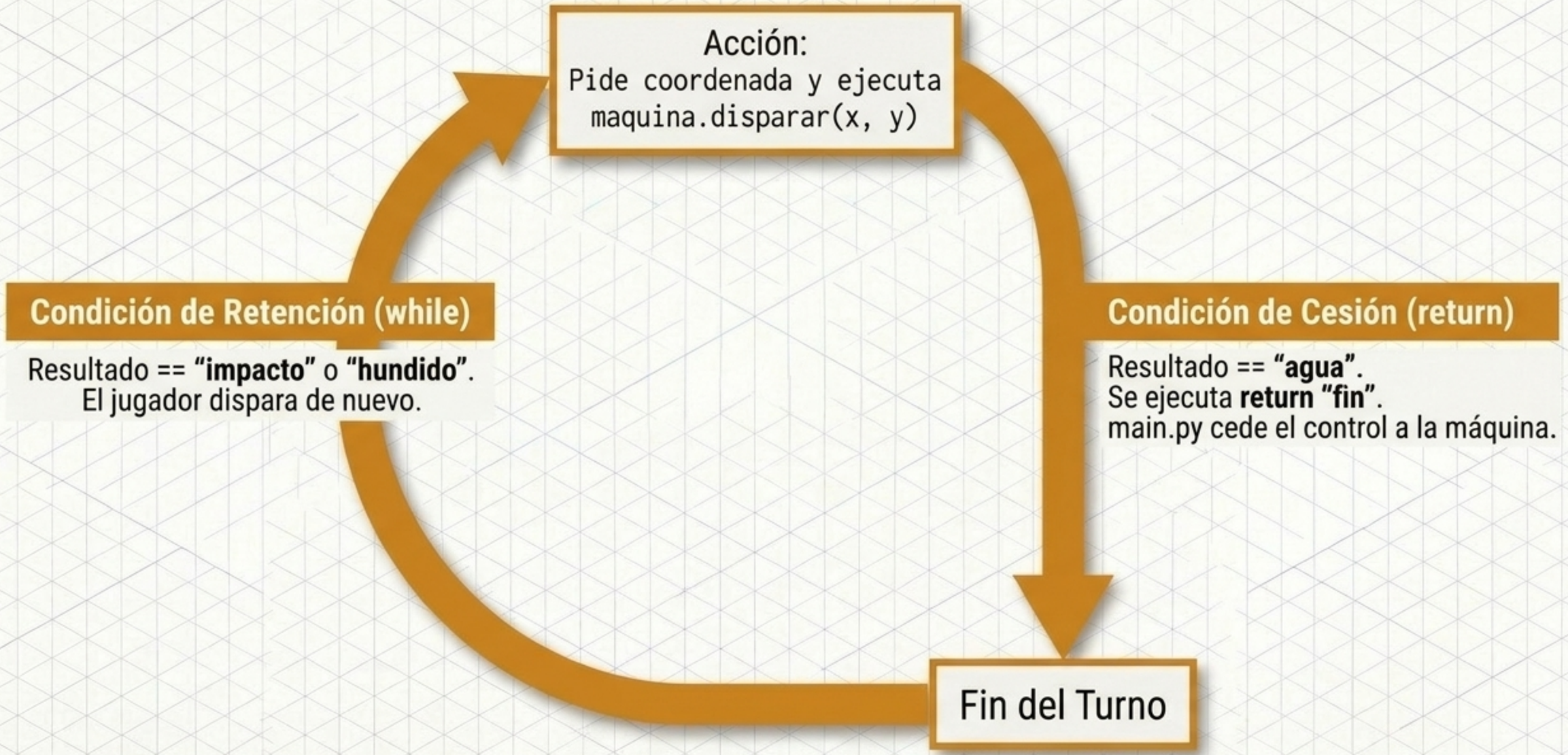
Gemini Notebook

Validación a Prueba de Balas (pedir_coordenadas)

Solo las entradas válidas logran llegar a la lógica de estado interno.

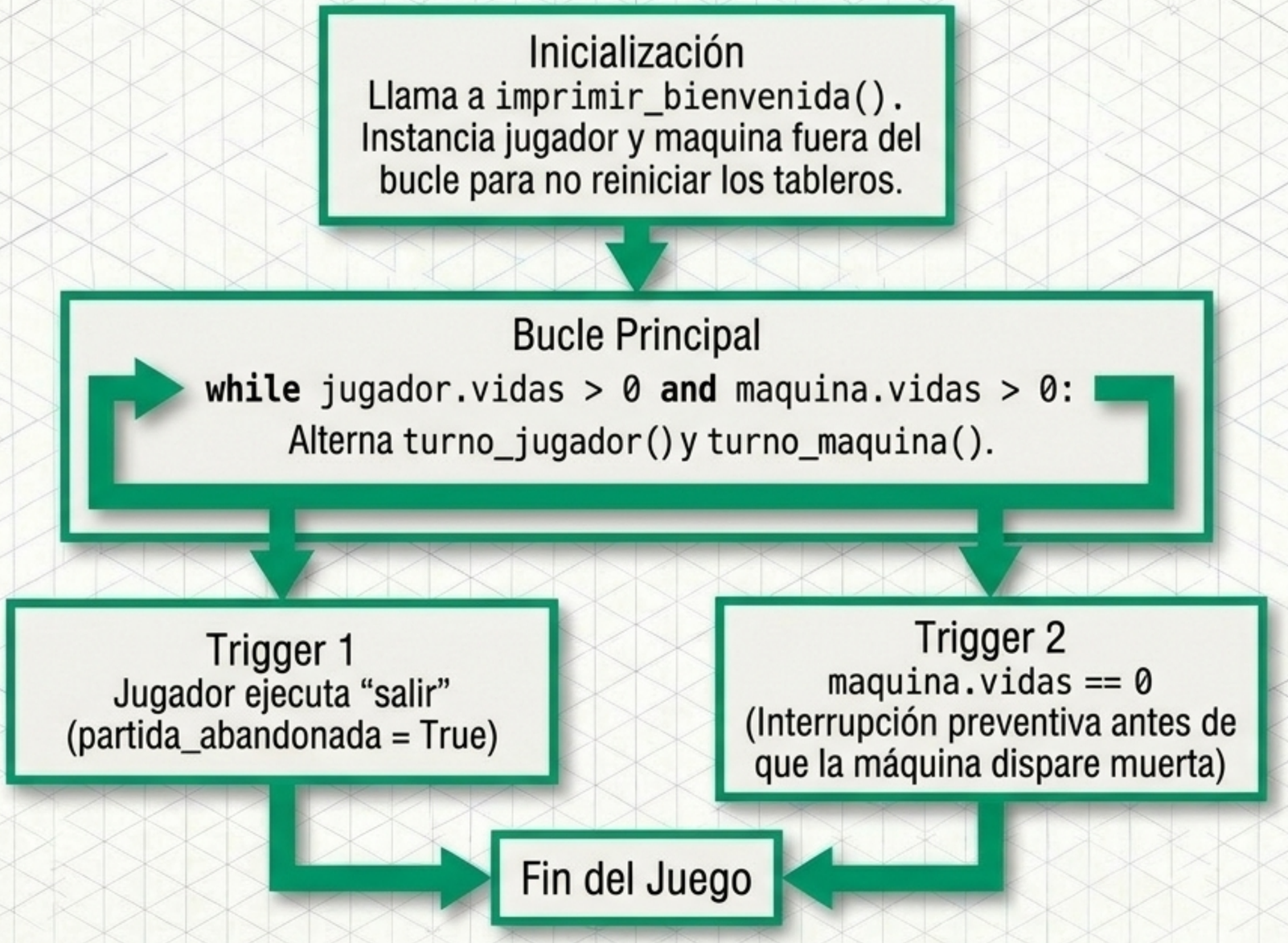
Gemini Notebook

La Anatomía del Turno: Retención y Cesión



Gemini Notebook

El Director de Orquesta (`main.py`)



Gemini Notebook

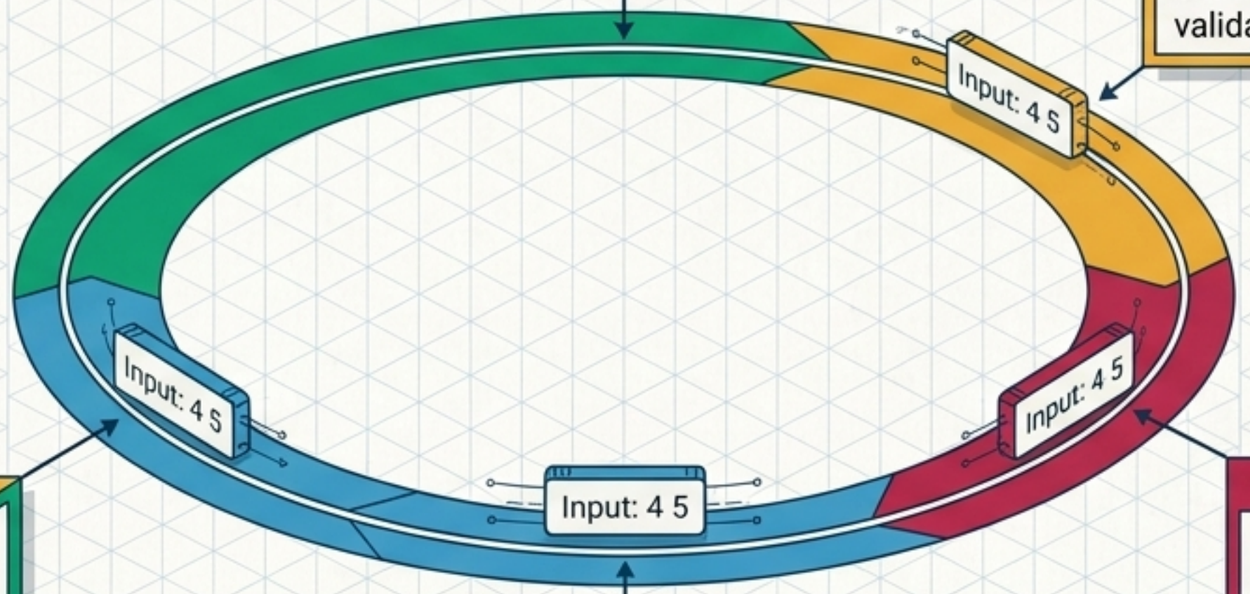
Síntesis: El Ciclo de Vida de un Disparo

Step 1

main.py solicita el inicio del turno del jugador.

Step 2

funciones.py recoge la entrada “4 5”, la limpia y valida los datos.



Step 3

clases.py recibe (4, 5), traduce al índice [4][3], y cruza la coordenada con las matrices internas.

Step 4

La lógica evalúa el impacto apoyándose en las constantes de **variables.py** (IMPACTO, BARCO).

Step 5

funciones imprime el resultado.

main actualiza vidas y reinicia el bucle.

Gemini Notebook

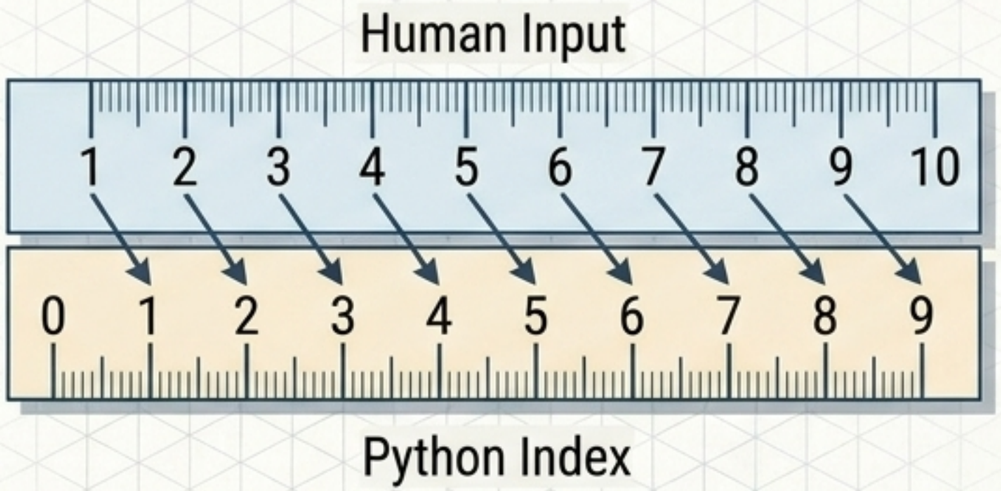
Matriz de Diagnóstico: Conceptos Clave de Python

	Función	Método
Estructura	Independiente (`def`). Se llama directamente (ej. `imprimir_bienvenida()`).	Pertenece a una clase. Requiere una instancia de objeto y el uso de `self` (ej. `jugador.mostrar_tablero()`).
Salida de Datos	**Print** Operación visual. Muestra información al humano por pantalla, pero el código subyacente no puede interactuar con ella.	**Return** Operación lógica. Devuelve un dato útil (ej. `'impacto'`) al programa para condicionar el flujo del juego.

Gemini Notebook

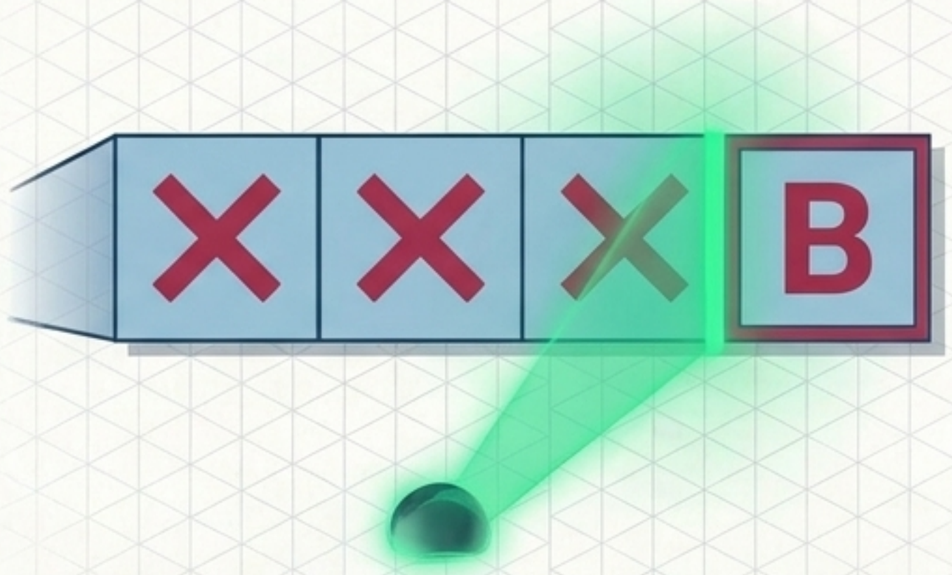
Superando la Complejidad (Lecciones Aprendidas)

Gestión de la Cuadrícula



Manejar listas anidadas de índice 0 frente a entradas humanas de índice 1 sin romper la ejecución.

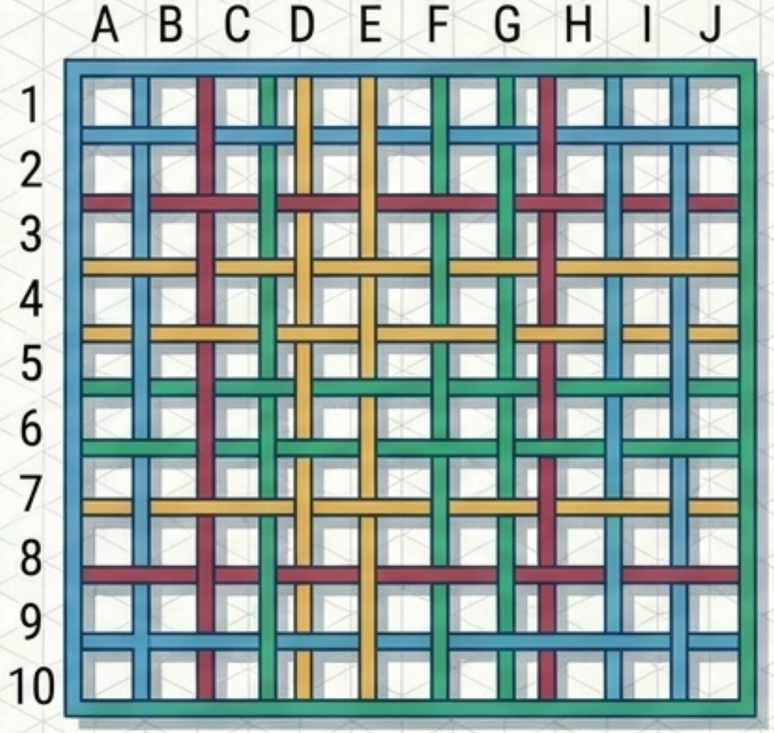
Flujo de Estados



Asegurar que la máquina identifique cuándo un barco individual está totalmente hundido (**iteración de tuplas**), más allá del simple contador de vidas globales.

Gemini Notebook

Un Sistema Limpio, Modular y Escalable



- ☞ Data/State Separation: **Aislar** las matrices de seguimiento de la flota **previene fugas de información.**
- ☞ Robust Input Validation: **Interceptar errores** antes de que toquen la lógica de estado **asegura la estabilidad.**
- ☞ Modular Scalability: Transformar la cuadrícula de 10×10 a 20×20 requiere cambiar una sola línea de código en `variables.py`.

Gracias. ¿Preguntas sobre el cód

Gemini Notebook